

Android 8.1&Android 9 OTA 升级指导手册

文档版本
发布日期

V1.4
2024-08-15

版权所有 © 紫光展锐（上海）科技有限公司。保留一切权利。

本文件所含数据和信息都属于紫光展锐（上海）科技有限公司（以下简称紫光展锐）所有的机密信息，紫光展锐保留所有相关权利。本文件仅为信息参考之目的提供，不包含任何明示或默示的知识产权许可，也不表示有任何明示或默示的保证，包括但不限于满足任何特殊目的、不侵权或性能。当您接受这份文件时，即表示您同意本文件中内容和信息属于紫光展锐机密信息，且同意在未获得紫光展锐书面同意前，不使用或复制本文件的整体或部分，也不向任何其他方披露本文件内容。紫光展锐有权在未经事先通知的情况下，在任何时候对本文件做任何修改。紫光展锐对本文件所含数据和信息不做任何保证，在任何情况下，紫光展锐均不负任何与本文件相关的直接或间接的、任何伤害或损失。

请参照交付物中说明文档对紫光展锐交付物进行使用，任何人对紫光展锐交付物的修改、定制化或违反说明文档的指引对紫光展锐交付物进行使用造成的任何损失由其自行承担。紫光展锐交付物中的性能指标、测试结果和参数等，均为在紫光展锐内部研发和测试系统中获得的，仅供参考，若任何人需要对交付物进行商用或量产，需要结合自身的软硬件测试环境进行全面的测试和调试。

商标声明

紫光展锐、**UNISOC**、、展讯、Spreadtrum、SPRD、锐迪科、RDA 及其他紫光展锐的商标均为紫光展锐（上海）科技有限公司及/或其子公司、关联公司所有。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

免责声明

本文档可能包含第三方内容，包括但不限于第三方信息、软件、组件、数据等。紫光展锐不控制且不对第三方内容承担任何责任，包括但不限于准确性、兼容性、可靠性、可用性、合法性、适当性、性能、不侵权、更新状态等，除非本文档另有明确说明。在本文档中提及或引用任何第三方内容不代表紫光展锐对第三方内容的认可、承诺或保证。

用户有义务结合自身情况，检查上述第三方内容的可用性。若需要第三方许可，应通过合法途径获取第三方许可，除非本文档另有明确说明。

紫光展锐（上海）科技有限公司



前言

概述

本文介绍展锐平台 Android 8.1、Android 9 的 OTA 升级过程，内容涵盖 OTA 简介、Recovery 系统介绍、升级包制作、升级测试、注意事项、Log 抓取方法等。

读者对象

本文档主要适用于进行 OTA 功能开发和测试的工程师，以及负责项目版本编制的工程师。

缩略语

缩略语	英文全称	中文解释
OTA	Over The Air	空中下载
TEE	Trusted Execution Environment	可信执行环境

符号约定

在本文中可能出现下列符号，每种符号的说明如下。

符号	说明
 说明	用于突出重要或关键信息、补充信息和小窍门等。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害。
 注意	用于突出容易出错的操作。 “注意”不是安全警示信息，不涉及人身、设备及环境伤害。
 警告	用于可能无法恢复的失误操作。 “警告”不是危险警示信息，不涉及人身及环境伤害。

变更信息

文档版本	发布日期	修改说明
V1.4	2024-08-15	删除 4.3 第三方 FOTA
V1.3	2024-07-02	更新 4.3 第三方 FOTA
V1.2	2023-08-02	更新 3.5.1 Otatools 工具获取
V1.1	2023-03-31	- 补充 UWS6137 的 Modem bins 改名映射 - 调整文档结构，优化文档描述
V1.0	2019-11-01	第一次正式发布

关键字

OTA、PAC 包、Recovery

目 录

1 OTA 简介	1
2 Recovery 系统介绍	2
2.1 启动脚本 init.rc	2
2.2 分区信息 fstab	2
2.3 显示资源 res	3
2.4 Recovery 主程序	3
3 升级包制作	6
3.1 升级包介绍	6
3.1.1 update-binary 升级程序	7
3.1.2 updater-script 升级脚本	7
3.2 制作升级包的原材料 target 包	8
3.3 升级包生成流程	10
3.4 升级包制作步骤	10
3.4.1 整体升级包制作步骤	10
3.4.2 差分升级包步骤	11
3.4.3 Modem bins 改名方法	12
3.5 通过 PAC 包产生 OTA 升级包	13
3.5.1 Otatools 工具获取	13
3.5.2 制作 OTA 包的命令介绍	13
4 升级测试	15
4.1 本地升级	15
4.2 GOTA	15
5 OTA 升级注意事项	17
5.1 NV 备份和 spl 升级	17
5.1.1 NV 备份	17
5.1.2 spl 升级	18
5.2 Secure Boot 使能的 OTA 升级	18
5.3 跨 Android 大版本升级	18
5.4 升级对 cache 空间要求	19
6 Recovery log 抓取办法	20
6.1 Userdebug 版本	20
6.2 User 版本	20

图目录

图 2-1 启动流程	4
图 2-2 Recovery 运行流程	5

1 OTA 简介

OTA (Over The Air, 空中下载) 业务是一项基于短消息机制, 通过设备终端或服务器方式实现 SIM 卡业务菜单的动态下载、删除与更新, 使用户获得个性化信息服务的数据增值业务。OTA 通过移动通信 (GSM 或 CDMA) 的空中接口对 SIM 卡数据及应用进行远程管理。空中接口可以采用 WAP、GPRS、CDMA1X 及短消息技术。OTA 技术的应用, 使移动通信不仅可以提供语音和数据服务, 还可以提供新业务下载。

OTA 升级包括编译、版本发布、终端版本检测、在终端上检测服务器最新版本、通过终端版本制作对应差分升级包、在终端下载服务器上的升级包、本地升级等一系列过程。

升级包放置到服务器, 终端设备的“系统升级”App 检测到升级包并下载到本地的过程主要由第三方的 FOTA App 完成。

本文档中介绍的 OTA 升级, 依靠 recovery 模式和升级包中的升级程序来完成。在升级过程中, Recovery 模式提供基本的功能, 包括挂载需要用到的分区到特定目录、输出 log 到 log 文件、检查电量是否足够、验证升级包是否完整正确、解析出升级程序和升级脚本并执行。其余升级有关的操作均由升级包中的 update-binary (升级程序) 完成。

2 Recovery 系统介绍

与正常启动的系统（以下简称 Android 系统）相比，Recovery 系统可以对设备系统做出一系列的更改，主要是恢复出厂设置和升级。

Android 与 Recovery 系统的异同点如下：

系统	异同点
Android 系统	包含的 img: - boot.img - Kernel: 正常编译时，相同。 - ramdisk: 不同，主要是 init.rc 不同。 - system.img - userdata.img
Recovery 系统	仅包含以下 img: - recovery.img - Kernel: 正常编译时，相同。 - ramdisk: 不同，主要是 init.rc 不同。

init.rc 的不同决定了 Kernel 在启动之后运行 Recovery 系统还是 Android 系统。

在系统启动进入 Recovery 模式后，可以从参数中获得升级包路径并进行升级，也可以在出现 Recovery 菜单后选择特定升级包进行升级。

Recovery 模式关键部分有：init.rc、recovery.fstab、显示资源 res、Recovery 主程序。

2.1 启动脚本 init.rc

Recovery.img 中的 init.rc 主要进行 recovery 模式的启动配置，包括简单的初始化、目录创建、服务启动，以及最主要的 recovery service。

init.rc 脚本位于 recovery 的 ramdisk 中，由 init 程序解释执行。在编译系统中，由 TARGET_RECOVERY_INITRC 来定义此脚本的存放目录。

2.2 分区信息 fstab

recovery.fstab 用于存放 Recovery 系统各个分区的信息。在制作升级包时，会用到此文件中的信息进行分区操作，在具体执行恢复出厂设置、升级时也会用到其分区信息。

Android 系统的 fstab 用于系统启动时 system、data 等分区的挂载以及存储设备的挂载、卸载、格式化等操作，仅包含 system、data、sdcard 等分区。而 Recovery 系统中的 fstab 用于存放所有可能需要操作的分区信息。这两种 fstab 的格式一致，但包含的分区不同。

在编译系统中，用变量 TARGET_RECOVERY_FSTAB 来定义实际使用的 recovery.fstab。

2.3 显示资源 res

Recovery 的显示采用 minui（与关机充电、工程模式等类似），使用的图片资源由设备的显示分辨率确定，根据 PRODUCT_AAPT_PREF_CONFIG 定义来选取。比如此值为 xhdpi，则会将 bootable/recovery/res-xhdpi/images 中的图片编译进 recovery.img，用于显示。

2.4 Recovery 主程序

Recovery 主程序主要对 Recovery 系统的流程进行控制、升级或恢复出厂设置。其编译在 bootable/recovery/Androd.mk 中定义。Recovery 模式的启动流程以及进入 Recovery 后的运行流程如图 2-1 图 2-2 所示。

图2-1 启动流程

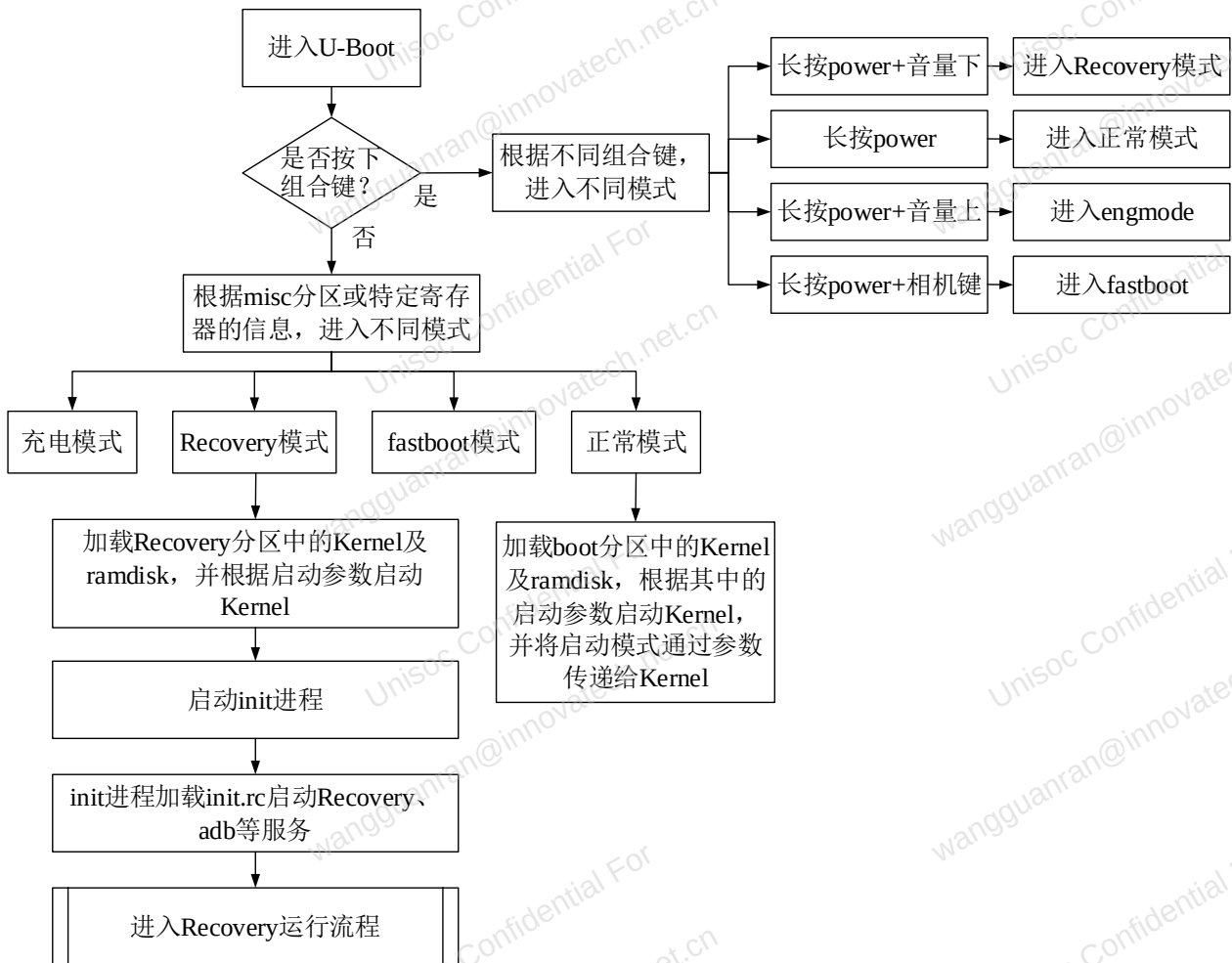
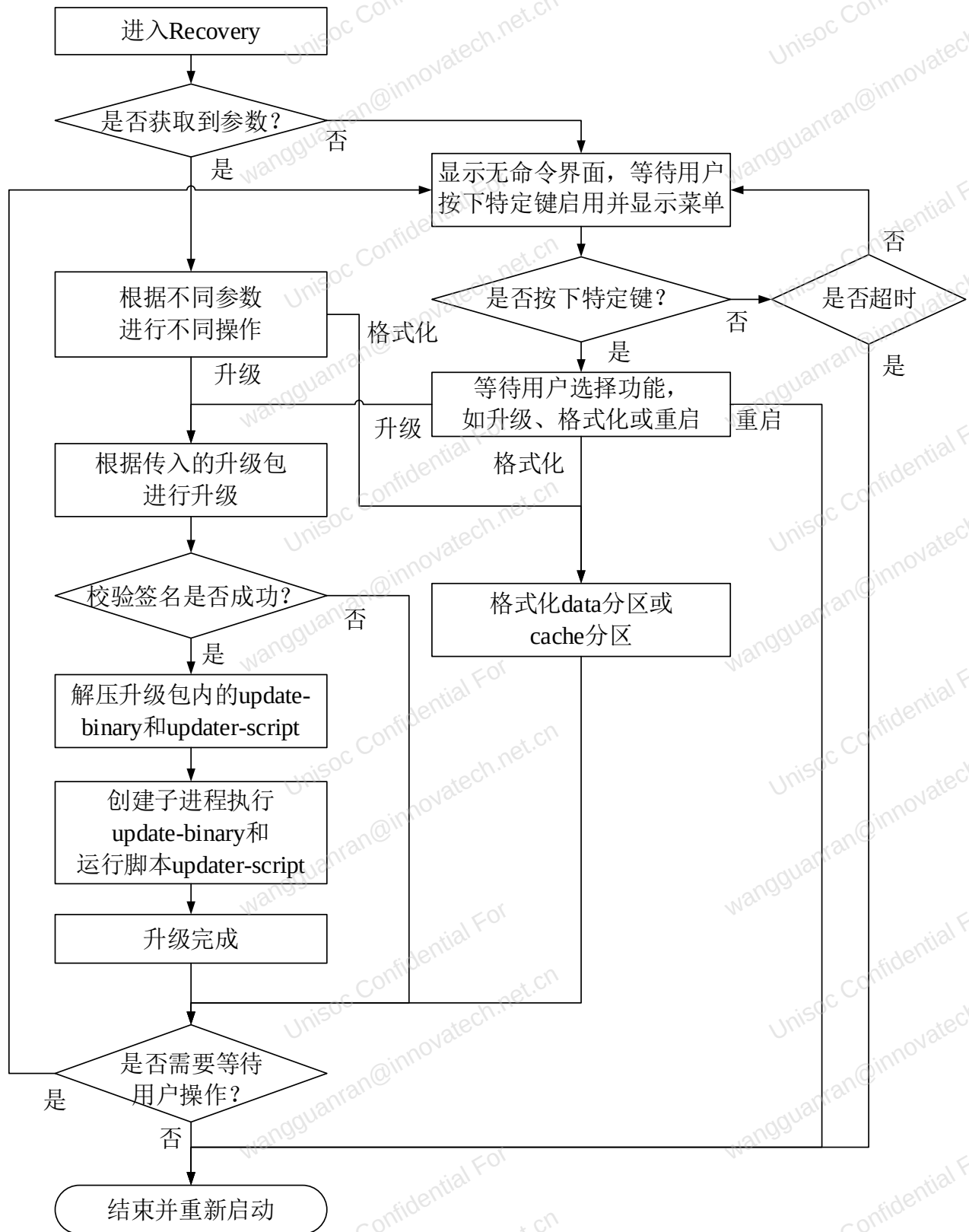


图2-2 Recovery 运行流程



3 升级包制作

3.1 升级包介绍

按照制作方式来划分，升级包分为整体升级包和差分升级包。

Recovery 系统在使用升级包升级时，创建了一个进程去执行 `update-binary`。升级包中还包含一个起主导作用的 `updater-script`。`update-binary` 只是一个解释器，用于执行 `updater-script` 中的命令。

System 分区采用块升级时的升级包文件列表：

—— dtb.img	目标版本dtb
—— dtbo.img	目标版本dtbo
—— gnssbdmodem.bin	目标版本modem bin
—— gnssmodem.bin	
—— ltedsp.bin	
—— ltegdsp.bin	
—— ltemodem.bin	
—— ltenvitem.bin	nv bin
—— META-INF	
—— com	
—— android	
—— metadata	写入升级结果的源目标版本信息
—— otacert	升级包签名的密钥
—— google	
—— android	
—— nvmerge	NV备份程序
—— nvmerge.cfg	NV备份列表
—— update-binary	升级程序
—— updater-script	升级脚本
—— patch	
—— boot.img.p	bootimg的差分patch

└── pmsys.bin	目标版本pmsys
└── product.new.dat	目标版本product分区新数据
└── product.patch.dat	目标版本product分区patch数据
└── product.transfer.list	目标版本product分区升级动作表
└── sml.bin	sml分区bin
└── system.new.dat	目标版本system分区新数据
└── system.patch.dat	目标版本system分区patch数据
└── system.transfer.list	目标版本system分区升级动作表
└── tos.bin	目标版本tos
└── u-boot.bin	目标版本uboot
└── u-boot-spl-16k.bin	目标版本spl
└── vbmeta.img	目标版本vbmeta
└── vendor.new.dat	目标版本vendor分区新数据
└── vendor.patch.dat	目标版本vendor分区patch数据
└── vendor.transfer.list	目标版本vendor分区升级动作表
└── wcnmodem.bin	目标版本wcnmodem

3.1.1 update-binary 升级程序

update-binary 升级程序在升级包中，路径为/META-INF/com/google/android/update-binary。

Recovery 系统在升级时，会将 update-binary 从升级包中解压到/tmp，并创建子进程执行此程序。

update-binary 来自源代码中的 updater 程序，包括写分区、解压文件、删除文件、执行程序等操作。

updater 程序负责解释和运行升级脚本/META-INF/com/google/android/updater-script，升级脚本中包含本次升级的内容。

Android 8.1 和 Android 9 上查看 bootable/recovery/updater/Android.mk，可知 updater 程序的源码包含如下文件：

```
updater_src_files := \
    install.cpp \
    blocking.cpp \
    updater.cpp
```

展锐独有的一些升级动作被解耦放到 vendor/sprd/tools/ota/sprd_updater 目录下，并在编译时被连接到 updater 程序中。

3.1.2 updater-script 升级脚本

updater-script 升级脚本在升级包中，路径为/META-INF/com/google/android/updater-script。

升级脚本中列举了升级时要执行的动作，包括对升级基准版本和时间戳产品名的校验、各分区的校验、各分区的升级、升级进度显示等。

此脚本是在升级包制作过程中由打包脚本生成的。打包脚本在决定各分区如何升级以及获得各分区数据后，把这些数据的具体升级动作写入到此脚本文件中。其具体的操作文件是 build/tools/releasetools/edify_generator.py。

3.2 制作升级包的原材料 target 包

原材料即版本对应的 target 包，其编译命令为：

```
make target-files-package
```

target 包编译过程主要就是将版本对应的各个分区的数据，以及这些分区的信息、升级时需运行的各个可执行程序放入 target 包中。

常见的 target 包目录结构：

target-files/		
——	BOOT	bootimg所有要素
——	base	Kernel基址
——	cmdline	传入Kernel的cmdline
——	kernel	zImage
——	pagesize	Kernel页大小，一般为2048
——	RAMDISK	正常启动时的ramdisk
——	DATA	data分区所有文件
——	dt.img	dt
——	IMAGES	版本对应各img
——	boot.img	
——	cache.img	
——	recovery.img	
——	system_image_info.txt	
——	system.img	
——	system.map	system块映射
——	userdata.img	
——	META	
——	apkcerts.txt	各App对应的签名key信息
——	boot_filesystem_config.txt	bootimg中ramdisk文件权限信息

		file_contexts.bin	selinux规则文件
		filesystem_config.txt	system分区所有文件权限信息
		misc_info.txt	整个系统关键信息
		otakeys.txt	ota升级包签名key信息
		recovery_filesystem_config.txt	Recovery的ramdisk中文件权限信息
		releasetools.py	展锐嵌入的打包脚本
		vendor_filesystem_config.txt	vendor分区所有文件权限信息
		OTA	
		android-info.txt	
		bin	升级中要运行可执行程序集
			modem.cfg 各Modem分区升级配置
			nvmerge NV备份程序
			nvmerge.cfg NV备份列表
			pack 备份程序
			repart 重新分区程序
			splmerge splmerge程序
			updater 升级程序
		RADIO	
			check.bin sysinfo分区bin
			ltdsp.bin modem bin
			ltegdsp.bin
			ltemodem.bin
			ltenvitem.bin
			ltewarm.bin modem bin
			partition.xml 版本分区表
			persist.img
			pmsys.bin modem bin
			sysinfo.img
			u-boot.bin uboot bin
			u-boot-spl-16k.bin spl bin
			wcnfdl.bin modem2 bin

		wcnmodem.bin	wcnmodem bin
		RECOVERY	
		base	Kernel基址
		cmdline	启动Recovery时传入Kernel的cmdline
		kernel	zImage
		pagesize	Kernel页大小
		RAMDISK	Recovery模式的ramdisk所有文件
		SYSTEM	system分区所有文件

3.3 升级包生成流程

打包脚本对 target 包进行加工，最终输出 OTA 升级包。打包脚本在 build/tools/releasetools，主入口是 ota_from_target_files。

打包脚本生成升级包的流程如下：

1. 读取传入脚本的参数、解压 target 包、创建 OTA 升级包临时文件。
2. 嵌入展锐特有的处理脚本 releasetools.py。
3. 获取 bootimg 数据并产生升级动作。
4. 获取 system 数据并产生升级动作。
5. 获取 recovery 数据并产生升级动作。
6. 运行展锐嵌入脚本 releasetools.py，进行 modem bins 和 uboot bins 升级动作处理。
7. 产生版本间 metadata 并写入升级包。
8. 对制作完的升级包进行签名。

3.4 升级包制作步骤

3.4.1 整体升级包制作步骤

以 SC9863A 为例，步骤如下：

1. 下载项目 AP 部分的代码。
2. 设置编译环境：

```
source build/envsetup.sh lunch kheader
```
3. 通过 make 命令全编整个工程。
4. 进入 device/sprd/sharkl3/s9863axxx/（s9863axxx 表示对应 board）目录，手动建立 modem_bins 子目录。

5. 将展锐发布的对应 AP 版本的 modem bins 按照 device/sprd/sharkle/s9863axxx/AndroidBoard.mk 中的规定，更改名字后复制到 device/sprd/sharkle/s9863axxxx/modem_bins/目录下。以下是 SC9863A 平台示例：

```
/device/sprd/sharkl3/s9863a10c10/AndroidBoard.mk
```

```
LOCAL_PATH := $(call my-dir)
```

```
$(call add-radio-file,modem_bins/ltemodem.bin)
```

```
$(call add-radio-file,modem_bins/ltegdsp.bin)
```

```
$(call add-radio-file,modem_bins/ltedsp.bin)
```

```
$(call add-radio-file,modem_bins/pmsys.bin)
```

```
$(call add-radio-file,modem_bins/ltenvitem.bin)
```

```
$(call add-radio-file,modem_bins/wcnmodem.bin)
```

```
$(call add-radio-file,modem_bins/ltedeltanv.bin)
```

```
$(call add-radio-file,modem_bins/gnssmodem.bin)
```

```
$(call add-radio-file,modem_bins/gnssbdmodem.bin)
```

modem bins 名字需依次改为 mk 文件中最后一列的名字，然后再进行复制，改名规则请参考 [3.4.3 Modem bins 改名方法](#)。

注意

主 Modem 以 “.dat” 为后缀的文件也要改为以 “.bin” 为后缀的文件。

6. 通过命令 “make otapackage” 编译 OTA 整包。此命令运行完后，会产生 target 包和 OTA 整包。整包目录：out/target/product/spxxx/spxxx-ota-*.zip。

注意

为了以后在版本升级时可以使用差分升级，请保留此版本对应的 target 包。路径为：out/target/product/spxxx/obj/PACKAGING/target_files_intermediates/*-target_files-*.zip

7. （可选）如果想制作跟 target 包对应的 PAC 包，请在此时执行命令生成 PAC 包。

注意

请务必在执行完 make otapackage 后做 PAC 包，因为 make otapackage 命令会对 boot.img 等进行重新编译，只有在此步骤后做的 PAC 包才与 target 包严格对应。

3.4.2 差分升级包步骤

1. 下载 A 版本代码，执行整包制作步骤 1~6，然后保存此版本对应的 target 包 A-target.zip。
2. 下载 B 版本代码，执行整包制作步骤 1~6，然后保存此版本对应的 target 包 B-target.zip。
3. 执行命令制作差分升级包。-k 后面的参数 sign_key_dir 为实际版本的 key 的放置目录，在 user 版本是 build/target/product/security/release/releasekey，在 userdebug 版本是 build/target/product/security/testkey。如果是块升级，则需加 --block 参数。编译命令如下：

- 正常差分包制作

```
./build/tools/releasetools/ota_from_target_files --block -k sign_key_dir -i A-target.zip B-target.zip A-B_update.zip
```

– 降级差分包制作

降级差分包指从新版本反向升级到老版本。Android 7.0 之后的版本支持此种升级包。由于回退升级中要擦除 userdata，**故此种升级包慎用**。这种差分包需要加入--downgrade 参数，此处须注意 A 版本必须要比 B 版本要新，即编译时间靠后。

```
./build/tools/releasetools/ota_from_target_files --downgrade --block -k sign_key_dir -i A-target.zip B-target.zip A-B__downgrade_update.zip
```

3.4.3 Modem bins 改名方法

本节以 SC9863A 和 UWS6137 的 Modem 改名规则为例。

3.4.3.1 SC9863A（Android 9）

Modem 原始名称	映射后命名
SC9600_sharkl3_pubcp_modem.dat	ltemodem.bin
sharkl3_pubcp_LTEA_DSP.bin	ltedsp.bin
sharkl3_pubcp_DM_DSP.bin	ltegdsp.bin
sharkl3_cm4.bin	pmsys.bin
gnssbdmodem.bin	gnssbdmodem.bin
gnssmodem.bin	gnssmodem.bin
PM_sharkl3_cm4.bin	wcnmodem.bin
sharkl3_pubcp_nvitem.bin	ltenvitem.bin
sharkl3_pubcp_Smart_Phone_deltanv.bin	ltedeltanv.bin

3.4.3.2 UWS6137（Android 8.1）

Modem 原始名称	映射后命名
sharkle_cm4.bin	pmsys.bin
sharkle_pubcp_Feature_Phone_W377_nvitem.bin	ltenvitem.bin
SC9600_sharkle_pubcp_Feature_Phone_modem.dat	ltemodem.bin
SharkLE_LTEA_DSP_evs_off.bin	ltedsp.bin
SHARKLE1_DM_DSP.bin	ltegdsp.bin
PM_sharkle_cm4_v2.bin	wcnmodem.bin
sharkle_pubcp_Feature_Phone_W377_deltanv.bin	ltedeltanv.bin
gnssbdmodem.bin	gnssbdmodem.bin
gnssmodem.bin	gnssmodem.bin

3.5 通过 PAC 包产生 OTA 升级包

PAC 包直接产生 OTA 升级包的优势：

- 无需进行 target 包编译，节省编译时间。
- 不会因 target 包丢失而无法产生升级包。
- 无需提供空间存储 target 包和 OTA 整包。

这种方法是从 PAC 包的 img 和分区表处解析出 target 包中制作 OTA 升级包必须的要素。其中有一些可执行程序是预制并放进升级包的。如果这些预制程序的源代码有改动，则需将更改代码后编出的程序替换进 device/sprd/sharkle/common/ota_binxxx/ (xxx 指芯片名)。

3.5.1 Otatools 工具获取

Otatools 工具不依赖代码，以工具包形式存在。此工具可以使用压缩和非压缩的 PAC 包产生升级包，能同时产生具备一定要素的 target 包或者 target 包文件目录，能产生 OTA 整包、差分包、降级包等。

Android 8.1

如需获取 Android 8.1 上的 Otatools 工具，请咨询展锐 FAE。

说明

由于产品更新换代，部分平台可能不支持使用 PAC 包产生 OTA 升级包。使用前请与展锐 FAE 进行确认。

Android 9

在 Android 9 上，此工具集成到代码中，只需执行命令即可在 \$OUT (out/target/product/xxx/, xxx 表示 board 的名字) 目录中获取 otatools，包括 otatools.zip 和 otatools 文件包，可以将 zip 文件或文件包放到其它路径直接使用。

源代码包产生 otatools 的命令，举例如下：

```
source build/envsetup.sh
lunch s9863a1h10_Natv-user-gms //此处建议使用user工程
kheader
make otatools //此命令即可产生工具包
```

注意

在自建 board 的自建工程中产生该工具时，不要更改 \$(PROPRIETARY_BOARD) 的定义。本工具会根据该变量值创建对应项目的目录。

3.5.2 制作 OTA 包的命令介绍

Otatools 工具使用方法请参见工具包中的 Guide.txt。此工具既能用 target 包产生 OTA 升级包，也能用 PAC 包产生 OTA 升级包。其中，使用 target 包产生 OTA 升级包的方法请参见 3.4 升级包制作步骤。

本节介绍使用 PAC 包产生 OTA 包的命令，如下所示：

```
./build/make/tools/releasetools/ota_from_pac_files [-d] [-r] <-p product_name> <-k sign_key> [-b pac_base] <-
```

```
t pac_target> <update_pkg>
```

制作 OTA 包的命令举例：

- 制作 OTA 整包

```
./build/make/tools/releasetools/ota_from_pac_files -p product_name -k sign_key -t pac_target full_update.zip
```

- 制作 OTA 差分升级包

```
./build/make/tools/releasetools/ota_from_pac_files -p product_name -k sign_key -b pac_base -  
t pac_target delta_base-to-target.zip
```

- 制作降级差分升级包

```
./build/make/tools/releasetools/ota_from_pac_files -d -p product_name -k sign_key -b pac_newer_build -  
t pac_oldrget delta_update_newer-to-old.zip
```

- 制作分区变更整体升级包

```
./build/make/tools/releasetools/ota_from_pac_files -r -p product_name -k sign_key -  
t pac_target full_update_repart.zip
```

制作 OTA 包的命令参数说明如下：

参数名称	说明
product_name	base 对应的项目名称，只能是小写。
sign_key	- user 版本：build/target/product/security/release/releasekey - userdebug 版本：build/target/product/security/testkey
pac_base	基准版本 PAC 包，支持.gz 的压缩格式和.pac 的非压缩格式。
pac_target	目标版本 PAC 包，同时支持压缩、非压缩格式。
full_update.zip	目标版本整包。
delta_base-to-target.zip	基准版本到目标版本的差分 OTA 升级包。
full_update_repart.zip	目标版本分区变更的 OTA 整包。

4 升级测试

展锐提供的 OTA 方案只支持本地升级，不支持实网升级的 FOTA App 及服务器端服务。

目前有两种渠道进行实网 OTA 升级：

- 使用 Google 的 GOTA
- 使用其他第三方方案的 FOTA

4.1 本地升级

本地升级主要验证 OTA 升级包是否能正常安装，这是最基础的升级调试。

按照 [3.4 升级包制作步骤](#) 制作出整包或差分包后，可以通过以下办法进行本地升级验证。

通过 Android 系统菜单进行升级

1. 将 OTA 升级包放到 SD 卡的根目录下并命名为 update.zip。

其中，Android 9 也支持内置存储本地升级，命令如下：

```
adb push update.zip /data/media/0
```

2. 依次点击“设置>关于>系统软件更新”，系统就会自动重启并升级。

直接进入 Recovery 模式升级

1. 将“升级包”放到 SD 卡或者 cache 分区根目录下。
2. 使设备置于关机状态。
3. 用组合键方式开机进入 Recovery 模式。

以手机为例，操作方法为：关机状态下，按 power 键 2~3 秒后再按住音量下键，亮屏后松开 power 键和音量下键，进入 Recovery 模式。

4. 根据“升级包”所在位置，选择相应选项，并选择升级包进行升级。
5. 升级完成后手动选择相应选项进行重启。

注意

SD 卡作为内部存储后，不能再作为本地升级的存储升级包的普通 SD 卡使用，因为其已经被挂载为 ext4 格式，而非 Recovery 能识别的 FAT 格式。

4.2 GOTA

GOTA 是使用 Google 服务器进行升级包部署，并使用 GMS 版本中内置的客户端进行升级包下载、校验、发起升级的一套实网升级服务。

升级流程如下：

1. 以 Android Partner 身份申请创建 GOTA Deployment 账户。
2. 由账户所有人配置对应的设备列表。
3. 由账户所有人上传升级包。
4. 由账户所有人配置升级包细节。
5. 设备检测升级包，下载并升级。

具体请参见 Google 资料网址。

5 OTA 升级注意事项

5.1 NV 备份和 spl 升级

在展锐平台上，有一些特殊分区，在烧录或者升级时不能将编译好的文件直接写入分区，需要进行一定的处理。nvmerge 和 splmerge 对 NV 分区和 spl 分区进行升级。

在升级脚本 updater-script 中的调用如下：

```
merge_spl("/cache/u-boot-spl-16k.bin", "/dev/block/mmcblk0boot0", "EMMC");  
assert(run_program("/tmp/nvmerge", "/tmp/nvmerge.cfg", "/dev/block/platform/sprd-sdhci.3/by-name/wfixnv1", "/cache/wnvitem.bin", "/cache/merged_wnvitem.bin", "0x40000"));  
write_emmc_image("/cache/merged_wnvitem.bin", "/dev/block/platform/sprd-sdhci.3/by-name/wfixnv1");  
write_emmc_image("/cache/merged_wnvitem.bin", "/dev/block/platform/sprd-sdhci.3/by-name/wfixnv2");
```

5.1.1 NV 备份

NV 备份过程：

1. NV 备份程序 nvmerge 扫描 NV 备份列表 nvmerge.cfg 中的要备份的 items。
2. nvmerge 把设备中 NV 分区的待备份 items 读取出来。
3. nvmerge 把这些 items 写入到目标版本的 nv bin 里面对应的 items 里面。
4. 所有要备份的 items 写完后，将经过组合的新的 nv bin 写入到分区里面。

OTA 升级时默认备份校准数据和 IMEI 等信息。如果还要备份别的数据，需在 nvmerge.cfg 中加入对应 items，比如加入 simlock 相关 items。

nvmerge 使用注意事项：

- 新版本（指升级后的目标版本）中的 NV 如果新增一些 items（不在默认备份列表之列），不影响 nvmerge 的结果，这时候新增的 items 都会体现在新版本的 NV 里面，不会影响新增 items 对应的功能。
- 新版本中的 NV 如果更改了一些不在默认备份列表中的 items 的结构（大小随之改变），不影响 nvmerge 结果。比如 audio 参数的增减。这时改变的 items 都会体现在新版本的 NV 里面，不会影响更改的 items 对应的功能。
- 新版本中的 NV 如果更改了在备份列表中 items 中的内容（名字结构无变化，只是更改对应值），则 nvmerge 会成功，但是新版本中这些更改过的值会被设备中现有版本中的值覆盖掉。
- 新版本中的 NV 如果更改了在备份列表中的 items 的结构（大小随之改变），则 nvmerge 会失败，结果是整个 NV 不会进行升级。

5.1.2 spl 升级

在 Android 8.1 上，编译产生的 spl bin 不能直接写入 spl 分区，而是需要将头部的校验信息根据 OTA 升级包中的 spl bin 计算填充完成后才能写入。splmerge 负责完成写入工作。

在 Android 9 上，不再需要 splmerge。升级 spl 的策略是直接将编译生成的 spl bin 写入分区即可，编译过程中已经填充好头部。

5.2 Secure Boot 使能的 OTA 升级

Secure Boot 使能后，OTA 制作的升级包中包含的 bin 也需要包含对应的签名。

展锐的 OTA 打包方案对 img 的签名处理、升级中对各 img 校验处理随着 Secure Boot 方案的演化也进行了相应的演化。下面以 Android 9 SC9863A 平台为例进行说明：

1. OTA 打包方案对 img 的签名处理

Android 9 SC9863A 平台 Secure Boot 一般默认使能，只有部分小内存工程没有默认使能。各 img、bin 的签名在编译完成后自动调用签名脚本进行签名，打 target 包时直接复制各签名好的 img、bin，modem bins 在 target 包中已经完成签名，OTA 打包过程中没有签名动作。因此无需做什么特殊动作，编出的 OTA 升级包已经包含合格签名的 img。

2. 升级过程中是否对 Secure Boot 签名进行校验

Android 9 SC9863A 平台在 Recovery 升级过程中将各 bin 写到分区前不再进行签名校验。如果写入非法签名的 bin，则无法开机。

5.3 跨 Android 大版本升级

Android 8.1 升级到 Android 9

Android 8.1 与 Android 9 存在以下差异：

- Android 9 上 Secure Boot 对分区表的 UID 数据新增了校验。
- Android 9 新增了分区。
- Android 8.1 与 Android 9 的 userdata 采用不同加密方式。

要想实现 Android 8.1 升级到 Android 9 时 userdata 不丢失，需进行以下对齐动作：

1. Android 8.1 基础软件版本分区必须和 Android 9 保持对齐，新增 dtb/dtbo（各 8M）、product 分区。
2. Android 8.1 基础软件版本需要经过产线写入分区 UID。
3. Android 8.1 基础软件版本在 TEE（Trusted Execution Environment，可信执行环境）中实现 Keymaster/Keystore。
4. Android 9 版本 userdata 的加密方式必须和 Android 8.1 保持一致，即 Android 9 的加密方式由 FBE 改为 FDE。
5. Android 9 目标软件版本的 Google security patch 版本一定要比 Android 8.1 高。

5.4 升级对 cache 空间要求

非 A/B 系统在 OTA 升级时必须提供 cache 空间，用于升级时缓存升级文件。cache 空间大小在不同场景下有所不同。在升级包制作过程中会检查 cache 空间是否足够。

评估确定 cache 分区大小的说明如下：

- 如果按文件升级，则比较 booting、modem bins 和 system 最大文件的大小，取最大值，cache 空间定为最大值的 1.2 倍或者+20MB 即可。一般来讲，除了一些 featurephone 项目，system 最大文件比 booting 和 modembins 大得多，因此一般以 system 最大文件值为准确定 cache 空间。
- 如果按块升级，booting 和 modem bins 升级所需 cache 空间一般小于 system 升级所需空间，因此确定 cache 空间时只需考虑 system 分区升级空间。版本差别大小、升级包大小、cache 大小之间存在连环效应。版本差别大，则所需 cache 空间大。如果 cache 空间不够，则采取增大升级包办法来保证升级成功。升级包大小一般为 150MB 或者 200MB。因此，需尽量避免两个差别巨大的版本间的差分升级，这样较小的升级包和 cache 就能升级成功。

综上，OTA 升级中的 cache 分区大小要求为：

- 在 system 按文件升级时，cache 空间比 system 最大文件稍大即可。
- 在 system 按块升级时，cache 空间保持与展锐基线大小一致。版本间不要差别太大，同时 cache 不能太小。

6 Recovery log 抓取办法

在恢复出厂设置或进行 OTA 升级时，可以按照以下方法抓取 Recovery 的 log。

6.1 Userdebug 版本

- 如果恢复出厂设置或 OTA 升级出错，并且重启后无法开机，则可在 Recovery 模式下选择“mount system”菜单并连接 adb。通过如下命令获取 Recovery 的 log：

```
adb root
adb pull /tmp/recovery.log D:\log
```

D 盘 log 下的 recovery.log 就是所需要的 log。

- 如果恢复出厂设置或 OTA 升级完成并重启成功，通过如下命令获取 log：

```
adb root
adb pull /cache/recovery D:\log
```

D 盘 log 里面的各文件就是 Recovery log。

6.2 User 版本

由于 user 版本无法连接 adb，也没有 root 权限访问 cache 分区，在没有对应的 userdebug 版本用于 debug 时，可以通过如下办法获取 log。

- 使用目标版本 PAC 包读取设备的 cache 分区数据并解析出 Recovery log。

注意

设备中 base 版本的分区表与 downloader 加载的 PAC 包中的分区表必须严格一致，否则读取的 cache 数据不对，无法解析出 log。

- Android 5.1 以后，平台在 Recovery 的菜单里有“view recovery log”，选择 cache/recovery/last_log，逐页查看，看到有出错语句的前后几页，拍照作为 log 图片。
- Android 7.0 以后，系统会将升级的 recovery log 复制一份到 SD 卡，user 版本的升级 log 可以直接从 SD 卡取出以供检查。Android 8.1、Android 9 的存储目录是 G:\recovery_log（此处假定 SD 卡在电脑上识别的盘符是 G）。